Coding Questions:

Question 1

Vehicle Rental System

Task

Design and implement a simplified vehicle rental system. The system should allow customers to rent available vehicles. A vehicle cannot be rented if there is an existing active rental for it. Different categories of vehicles (e.g., Sedan, SUV, Truck) may have different rental charge calculation rules. The total rental amount depends on the selected vehicle type and the duration of the rental. Once a vehicle is returned, it must become available for new rentals.

Your solution should:

1. Identify key classes (e.g., Vehicle, Customer, Rental) and their attributes and operations. 2. Model the relationships between these classes, including multiplicity and any applicable inheritance or interface relationships to handle different vehicle categories and their distinct rental charge rules. 3. Implement the core workflows for renting a vehicle, calculating the rental charge, and returning a vehicle. 4. Ensure that the system supports the introduction of new vehicle categories without modifying the common rental processing logic.

Sample Input: Customer 1 rents Sedan A for 3 days. Customer 2 attempts to rent Sedan A for 2 days (while it's rented by Customer 1). Customer 1 returns Sedan A. Customer 3 rents SUV B for 5 days.

Sample Output (expected behavior): Sedan A rented successfully by Customer 1. Rental charge: $X. Sedan A is currently unavailable. Sedan A returned by Customer 1. SUV B rented successfully by Customer 3. Rental charge: $Y.

Instructions

Consider using an abstract base class or an interface for Vehicle to define common behavior and allow specific vehicle categories to implement their own pricing logic. The Rental object should track the rented vehicle, customer, and rental period. Think about how to manage the availability status of vehicles.

Starter code

```
// Implement: Vehicle Rental System
```

Question 2

Employee Leave Request Workflow

Task

Design and implement an employee leave management system. Employees can submit leave requests specifying start and end dates. Each leave request has a status (e.g., Pending, Approved, Rejected). All requests must undergo a review process before they can be approved or rejected. Once a request is Approved or Rejected, it cannot revert to the Pending state. Different types of employees (e.g., FullTimeEmployee, PartTimeEmployee, Contractor) may have distinct leave policies or approval workflows.

Your solution should:

1. Identify the main classes, such as Employee, LeaveRequest, and potentially LeavePolicy or Reviewer. 2. Define their attributes and methods, ensuring appropriate access control for managing leave request states. 3. Model the relationships between these classes, including any generalization/specialization for different employee types and their leave rules.
4. Implement the core functionality for submitting a leave request and the review process (approving or rejecting requests), including validation of state transitions.
5. Design the system such that adding a new employee type with different leave rules does not require changes to the existing leave processing workflow.

Sample Input: FullTimeEmployee John submits leave request for 5 days (Jan 1-5). Manager Alice reviews John's request and approves it. PartTimeEmployee Jane submits leave request for 2 days (Feb 10-11). Manager Bob reviews Jane's request and rejects it. John attempts to change his approved leave request to Pending.

Sample Output (expected behavior): Leave request submitted for John (Jan 1-5). Status: Pending. John's leave request (Jan 1-5) approved. Status: Approved. Leave request submitted for Jane (Feb 10-11). Status: Pending. Jane's leave request (Feb 10-11) rejected. Status: Rejected. Cannot change leave request status from Approved to Pending.

Instructions

Consider using an abstract class or interface for Employee to define common behaviors and allow subclasses to implement specific leave rule checks. The LeaveRequest class should manage its state transitions and guard against invalid changes. Think about how to separate the responsibility of leave policy enforcement from the core leave request management.

Starter code

```
// Implement: Employee Leave Request Workflow
```

Question 3

Online Examination System

Task

Design and implement an online examination system. A student can start an examination, answer multiple questions, and submit it. An examination contains multiple questions, which can be of different types (e.g., MultipleChoiceQuestion, TrueFalseQuestion, ShortAnswerQuestion), each requiring different evaluation logic. A student can only have one submitted attempt for a particular examination. Once an attempt is submitted, its answers cannot be changed. The system must calculate the result for each question and the overall score after submission.

Your solution should:

1. Identify the core classes such as Student, Examination, Question, Attempt, and Answer. 2. Define the attributes and operations for these classes, including how questions are stored and how answers are recorded.
3. Model the relationships, multiplicities, and any inheritance/interface relationships necessary to support different question types and their unique evaluation behaviors.
4. Implement the workflow for starting an examination, recording answers, submitting the examination, and calculating the final result.
5. Ensure that the design supports adding new question types in the future without altering the main examination submission and evaluation logic.

Sample Input: Student 1 starts Exam A. Student 1 answers Question 1 (MCQ) with option C. Student 1 answers Question 2 (TF) with True. Student 1 submits Exam A. Student 1 attempts to change answer for Question 1.

Sample Output (expected behavior): Exam A started by Student 1. Answer recorded for Question 1. Answer recorded for Question 2. Exam A submitted by Student 1. Result: Question 1: Correct (5 points), Question 2: Incorrect (0 points). Total score: 5/10. Cannot change answers for a submitted examination.

Instructions

Use an abstract base class or an interface for Question to define common question behavior and a method for self-evaluation. Each specific question type can then implement its evaluation logic. The Attempt object should track the student, the examination, and their recorded answers, and manage its state (e.g., InProgress, Submitted).

Starter code

```
// Implement: Online Examination System
```

Question 4

Hotel Booking System

Task

Design and implement a hotel booking system. Customers can reserve available rooms. A room cannot have overlapping active reservations. Different room categories (e.g., StandardRoom, DeluxeRoom, Suite) may utilize different pricing rules based on amenities or size. A reservation can only be canceled before a specific cancellation deadline. The booking amount must be dynamically calculated based on the selected room category and the duration of the reservation.

Your solution should:

1. Identify the core classes such as Customer, Room, Reservation, and their attributes and operations.
2. Define how room availability is managed and how reservations are made.
3. Model the relationships between these classes, including multiplicities and any inheritance/interface relationships to accommodate diverse room categories and their pricing rules.
4. Implement the key functionalities: checking room availability for a given period, creating a reservation, calculating the reservation price, and canceling a reservation (with cancellation deadline enforcement).

Sample Input: Customer A checks availability for Standard Room 101 from Jan 1 to Jan 5. Customer A reserves Standard Room 101 from Jan 1 to Jan 5. Customer B attempts to reserve Standard Room 101 from Jan 3 to Jan 7. Customer A cancels reservation for Standard Room 101 for Jan 1-5 (before deadline). Customer C reserves Deluxe Room 201 from Feb 10 to Feb 12.

Sample Output (expected behavior): Standard Room 101 is available from Jan 1 to Jan 5. Reservation confirmed for Customer A, Standard Room 101 (Jan 1-5). Price: $X. Standard Room 101 is not available from Jan 3 to Jan 7. Reservation for Customer A, Standard Room 101 (Jan 1-5) cancelled successfully. Reservation confirmed for Customer C, Deluxe Room 201 (Feb 10-12). Price: $Y.

Instructions

Consider using an interface or an abstract class for Room to define common properties and an abstract method for calculatePrice(), allowing concrete room types to implement their specific pricing logic. The Reservation class should hold start and end dates and a reference to the Room and Customer. Availability checking needs to consider existing reservations.

Starter code

```
// Implement: Hotel Booking System
```

Question 5

Payment Processing for a Shopping System

Task

Design and implement a payment processing module for an online shopping system. A Customer places an Order which can contain multiple Products. An Order must have at least one item before payment can be attempted. The system supports various payment methods (e.g., CreditCardPayment, PayPalPayment, BankTransferPayment), and each method processes payment differently. An Order is marked as Paid only after a successful payment transaction. A failed payment must not change the Order's status to Paid.

Your solution should:

1. Identify the primary classes: Customer, Order, Product, and various PaymentMethod implementations.
2. Define their essential attributes and operations, including how products are added to an order and how payment is initiated.
3. Model the relationships between these classes, utilizing inheritance or interfaces to manage different payment methods polymorphically.
4. Implement the workflow for attempting payment, processing payment through a selected method, and updating the order status based on payment success or failure.
5. Ensure the system design allows for the seamless addition of new payment methods without requiring modifications to the existing order-processing or payment initiation logic.

Sample Input: Customer X creates an order with Product A (qty 2) and Product B (qty 1). Customer X attempts to pay for the order using Credit Card. Credit Card payment is successful. Customer Y creates an empty order. Customer Y attempts to pay for the empty order. Customer Z creates an order with Product C (qty 1). Customer Z attempts to pay for the order using PayPal. PayPal payment fails.

Sample Output (expected behavior): Order created for Customer X. Payment initiated via Credit Card for Order X. Payment for Order X successful. Order status: Paid. Cannot process payment for an empty order. Order created for Customer Z. Payment initiated via PayPal for Order Z. Payment for Order Z failed. Order status: Pending.

Instructions

Define an interface or an abstract class for PaymentMethod with a processPayment() method. Concrete payment methods will implement this. The Order class should manage its items and status. The payment processing logic should depend on the PaymentMethod abstraction, not its concrete implementations. Consider how the order's state changes based on payment outcomes.

Starter code

```
// Implement: Payment Processing for a Shopping System
```

Quiz question

Question 1

A ticketing system manages various event types like concerts, plays, and sports matches. While all events have a title and a scheduled date, concerts also have a performer, and sports matches have competing teams. The system processes tickets for any event type through a common TicketProcessor.

Which OOP concept allows the TicketProcessor to interact with different event types uniformly?

A Encapsulation
B Inheritance
C Polymorphism
D Aggregation

Question 2

In a social media application, a UserProfile object has a dateOfBirth attribute. The application prevents direct modification of dateOfBirth from outside the UserProfile class, requiring any changes to be made through a updateProfileInfo method that includes validation logic to ensure the date is valid and reasonable.

Which OOP principle is being primarily enforced here?

A Inheritance
B Abstraction
C Encapsulation
D Composition

Question 3

A publishing house manages authors and their books. An author can write one or more books, but a book must always have exactly one author. If the author is removed from the system, their books still exist but might be reassigned to another author or marked as unauthored.

Which UML multiplicity best describes the relationship from Author to Book?

A Author 1 --- 1 Book
B Author 0..* --- 1 Book
C Author 1 --- 1..* Book
D Author 1..* --- 0..* Book

Question 4

A Shopping Cart object directly manages a collection of Cart Item objects. If the Shopping Cart is deleted, all its associated Cart Item objects are also destroyed, as they have no meaning or independent existence outside of that specific cart.

Which UML relationship accurately models this strong ownership and lifecycle dependency?

A Association

B Aggregation
C Dependency
D Composition

Question 5

An online course platform defines a general Course concept. Specific types of courses, such as ProgrammingCourse and DesignCourse, extend this general Course by adding specialized attributes and behaviors while inheriting common ones. The specialized courses are essentially 'is-a' types of the general course.

Which UML relationship represents the connection between the general Course and ProgrammingCourse?

A Realization
B Dependency
C Aggregation
D Generalization

Question 6

A Printer class needs to interact with various Printable document types (e.g., PdfDocument, WordDocument). The Printer is designed to work with any object that adheres to a Printable contract, which defines a print() method, without needing to know the specific implementation details of each document type.

Which UML relationship describes the connection between PdfDocument and the Printable contract?

A Generalization
B Composition
C Realization
D Association

Question 7

A Customer places an Order. The Customer can exist without placing any orders, and an Order can be created and completed even if the Customer record is later archived (though it would still reference the customer ID). There is no strong ownership or lifecycle dependency between them; they are just related by a transaction.

Which UML relationship is most suitable for describing this scenario?

A Composition
B Association
C Generalization
D Dependency

Question 8

A task in a project management system has various statuses: Pending, InProgress, Completed, Blocked. A task can move from Pending to InProgress, from InProgress to Completed or Blocked, but a Completed task cannot return to Pending or InProgress. This behavior defines a specific flow of states and transitions.

Which UML diagram type is best suited to model the lifecycle and valid transitions of a Task object?

A Class diagram
B Sequence diagram
C State diagram

D Component diagram

Question 9

A reporting module generates various types of reports (e.g., Sales Summary, Inventory List, Employee Roster). The module interacts with a generic ReportGenerator interface, which defines a generate() method. This allows the module to request a report without needing to know the specific details of how each report type is created.

Which design concept is primarily demonstrated by using a generic ReportGenerator interface?

A Encapsulation
B Abstraction

C Composition
D Object identity

Question 10

When a user logs into an application, the following sequence of actions occurs: the user enters credentials, the AuthenticationService receives these credentials, it then calls the UserRepository to verify them, the UserRepository returns the result, and finally, the AuthenticationService grants or denies access. This interaction involves multiple objects exchanging messages over time.

Which UML behavioral model is most appropriate for visualizing this specific order and flow of messages between objects during the login process?

A Class diagram
B Activity diagram
C Sequence diagram
D Use case diagram

Concept Questions:

Question 1

A user registration system requires users to provide an email address. The User object has an email attribute. To ensure data integrity, the system prevents direct modification of the email attribute. Instead, any email changes must go through a changeEmail method that includes validation to check for a valid format and uniqueness before updating the email.Explain how encapsulation is applied in this User object design. Describe what state should be controlled by the User object, why unrestricted access to the email could lead to invalid states, and how controlled access through methods helps maintain valid object state.

Question 2

Consider a ReportingService that can generate various types of reports. Some reports might require DataExport capabilities (e.g., export to CSV), while others might require DataVisualization capabilities (e.g., generate charts). Not every report needs all capabilities. The ReportingService itself has a core set of functionalities.Explain when
inheritance might be considered and when composition would be more suitable for incorporating these optional DataExport and DataVisualization capabilities into the reporting system. Discuss how forcing all capabilities into a single inheritance hierarchy for all report types could lead to design rigidities and make the system harder to extend with new report features.

Question 3

An application has a NotificationService responsible for sending messages to users. It supports sending notifications via EmailNotification, SMSNotification, and PushNotification. The NotificationService should be able to trigger a notification without explicit conditional logic for each specific notification type.Explain how abstraction (e.g., an interface or abstract class), method overriding, and runtime polymorphism work together to achieve this flexible notification system. Describe what happens when a new notification implementation (e.g., InAppNotification) is introduced, and how the existing NotificationService remains largely unaffected.

Question 4

An Organization is composed of several Departments. Each Department has a collection of Employees. When an Organization ceases to exist, its Departments are also dissolved. However, if a Department is dissolved, its Employees might still exist and could be reassigned to other departments or become independent. Similarly, Employees can move between Departments throughout their career.Explain the differences between composition and aggregation based on this scenario. Justify which UML relationship is more appropriate for Organization and

Department, and which is more appropriate for Department and Employee, based on their respective ownership and lifecycle dependencies.

Question 5

An educational platform has Courses and Students. A Course can be taken by many Students, and a Student can enroll in multiple Courses. A specific business rule states that a student cannot be enrolled in the same course more than once, meaning there should be no duplicate enrollment records for a student-course pair.Explain the UML multiplicity that correctly describes the relationship between Student and Course in this scenario. Furthermore, clarify why the business rule preventing duplicate enrollment is distinct from, and not directly expressed by, the multiplicity itself, but rather enforced by other mechanisms in the system's logic.